# MOCASSIN Spatial Refinement: Patch-Based AMR vs. Octree

## 1. Motivation

MOCASSIN is a Monte Carlo photo-ionisation and dust radiative-transfer code built on a Cartesian grid with optional nested refinement. The current refinement strategy — a flat list of independent rectangular sub-grids linked by parent pointers — has served well for the historical use cases (planetary nebulae, isolated H II regions) but is ill suited to problems demanding adaptive resolution: deeply nested ionisation fronts, multi-scale dust geometries, or shock-driven structures whose location is unknown a priori. An octree replacement is therefore under consideration.

The purpose of this memo is to lay out the existing data structures and algorithms in enough detail that the necessary changes for an octree transition are visible at a glance, and to document where the equivalence between the two designs breaks down.

## 2. Current Structure: Patch-Based Nested Rectilinear AMR

### 2.1 Grid container

The active grid is held in a flat Fortran array,

```
type(grid_type) :: grid3D(maxGrids) ! mocassin.f90
integer :: nGrids ! number of grids in use
```

Each `grid_type` (defined in `common_mod.f90:267-350`) is a fully self-contained Cartesian grid: it owns its own resolution $(n_x, n_y, n_z)$, face-aware coordinate arrays `xFace`, `yFace`, `zFace`, an active-cell indicator $\mathcal{A}(i, j, k)$, and the entire suite of per-cell physical state arrays (`Hden`, `Ne`, `Te`, `ionDen`, `Jste`, `Jdif`, `opacity`, `escapedPackets`, …), which are stored as one-dimensional arrays of length `nCells` indexed by the active map.

The hierarchy is encoded by a single integer per grid, `motherP`, which points to the index of the parent grid in `grid3D`. The root grid has `motherP` = `1`; sub-grids carry `motherP` > `1`. The mother grid records the cell range $[i_0, i_1] \times [j_0, j_1] \times [k_0, k_1]$ that each child covers in `motherX`, `motherY`, `motherZ`.

## 2.2 Active map and child dispatch

The integer 3D array $\mathcal{A}(i,j,k)$ encodes three states in a single field:

$$\mathcal{A}(i,j,k) = \begin{cases} n > 0 & \text{cell is active; } n \text{ is the flat index into per-cell arrays,} \\ 0 & \text{cell is inactive (outside density cut),} \\ -g & \text{cell is covered by sub-grid } g. \end{cases} \tag{1}$$

Mother-grid cells overlapped by a sub-grid are marked with the negation of the child's grid index, so that traversal logic can dispatch with a single sign test.

## 2.3 Refinement specification

Refinement is *static* and *user-defined* at run time. The input file declares

```
multiGrids 2 'input/subgrid.in'
```

and `subgrid.in` contains, per child, a fixed-extent rectilinear patch:

```
<gridID> <nx> <ny> <nz> '<density.dat>' <fillingFactor>
<xmin> <xmax> <ymin> <ymax> <zmin> <zmax>
```

The user is responsible for choosing both the location and the resolution of each sub-grid; the code does not adapt at run time.

## 2.4 Photon traversal

Cell-to-cell ray marching uses Amanatides–Woo three-dimensional digital differential analysis (3D-DDA) on the face-aware coordinate arrays (`dda_mod.f90`). Within a single grid this is the textbook algorithm: at every step the algorithm advances along the axis that minimizes the parametric distance to the next face.

When a packet enters a mother cell whose active map satisfies $\mathcal{A}(i,j,k) < 0$, control transfers to the sub-grid via `enterSubgrid` (`subgrid_mod.f90`), which sets the new grid pointer $g_P \leftarrow |\mathcal{A}(i,j,k)|$ and re-locates the packet on the child's coordinate axes. Outward escape from a sub-grid is handled by `subgridOutwardCheck` (`multigrid_mod.f90`), which promotes the packet back to the root grid. The inner traversal loop bound is `nGrids+2`, a direct function of the maximum nesting depth.

## 2.5 MPI parallelism

Cells are distributed across MPI ranks by a flat round-robin index over the union of all grids:

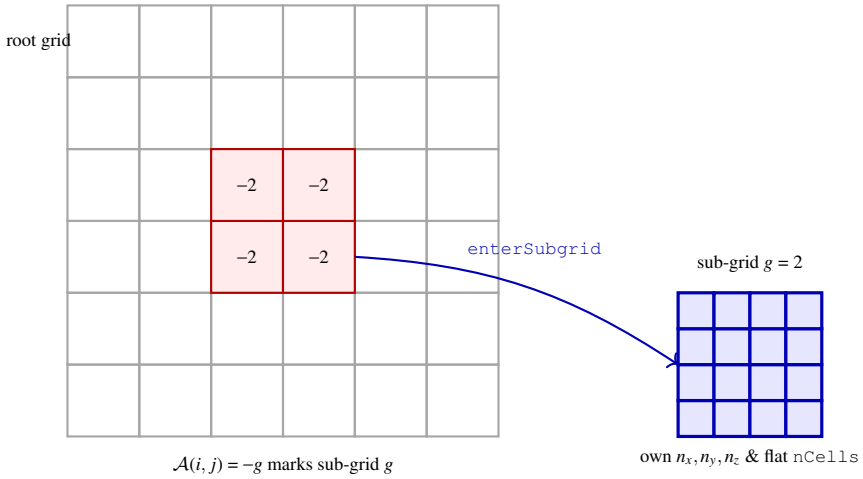
```
do iG = 1, nGrids
do i,j,k:
  iCell = iCell + 1
  if (mod(iCell - (taskid+1), numtasks) == 0) call ionizationDriver(...)
```

Sub-grid boundaries play no role in the partition. Hot-path arrays (`opacity`, `recPDF`, `absOpac`, `scaOpac`, `dustPDF`) have been migrated to MPI-3 shared-memory windows so that each compute node holds exactly one copy.

### 2.6 Output and restart

Grid topology is serialized to `grid0.out`: `nGrids` followed by, for each grid, the dimensions, cell count, mother-grid pointer, the outer radius, and the three coordinate axes. Per-cell physical state is written to `grid1.out` (gas) and `grid3.out` (dust), one contiguous block per grid. The restart driver `mocassinWarm` reconstructs the full hierarchy from these files alone.

### 2.7 Schematic




## 3. Octree Alternative

### 3.1 Topology

An octree is a recursive $2\times2\times2$ subdivision of the computational domain. The root encompasses the whole simulation volume; any node may be refined into eight axis-aligned children of equal size. Nodes are either *leaves* (carrying physical state) or *internal nodes* (carrying only topological information). Each cell sits in a uniquely defined leaf, and the level of refinement varies from leaf to leaf.

A common variant relaxes the "one cell per leaf" rule and instead stores a small uniform block, typically $4^3$ or $8^3$ cells, inside every leaf. This is known as a *block-structured octree* and is the design adopted by FLASH, Athena++, and ART. It trades a coarser refinement granularity for substantially better cache locality and DDA hot-path efficiency.

## 3.2 Recommended representation

A practical implementation combines two views of the same tree:

1. A *pointer-based* view for traversal,

```
type :: oct_node
  integer :: level
  integer(int64) :: morton_key
  real(dp) :: bbox(6) ! xmin..zmax
  integer :: parent
  integer :: child(8) ! 0 = leaf
  integer :: leaf_id ! index into per-leaf arrays
  integer :: owner_rank ! MPI ownership
end type
type(oct_node), allocatable :: oct_pool(:)
```

2. A *Morton-ordered linear* view of the leaves only, used for space-filling-curve (SFC) load balancing and for $\mathcal{O}(\log N)$ point location via binary search on the sorted Morton-key array.

## 3.3 Refinement criterion

Where the patch design relies on the user, the octree picks its leaves from a refinement function $\mathcal{R}(\text{leaf})$. Common choices are

$$\mathcal{R}(\text{leaf}) = \alpha \frac{|\nabla \log \tau|}{\tau_0} + \beta \frac{|\nabla \log n_{\text{H}}|}{n_0} + \gamma f_{\text{photons}}, \tag{2}$$

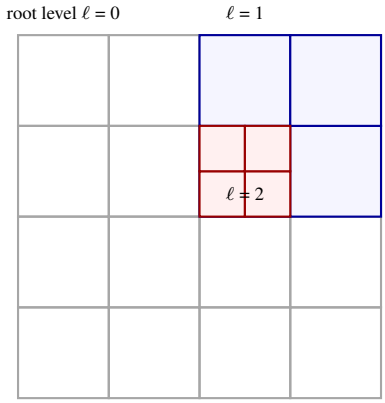
with thresholds for splitting and merging. At least in a first implementation it is reasonable to bypass adaptivity entirely and seed the octree from the user's existing patches: this preserves backwards compatibility while gaining the unified traversal infrastructure.

## 3.4 Ray traversal

Two well-known algorithms apply. The *parametric octree traversal* of Revelles, Ureña, and Lastra (2000) sorts the eight children of any internal node analytically and visits them in the order they are entered by the ray, without recursion. Stackless variants, in which one jumps directly between Morton-key neighbors via bit manipulation, are competitive when the tree is shallow and well balanced.

In a block-structured implementation the outer loop is octree descent — in the hot path, one descent per leaf crossing — and the inner loop is the existing 3D-DDA over the leaf's local $4^3$ or $8^3$ cells. This keeps the fast inner loop almost identical to the present code.

### 3.5 Schematic




## 4. Side-by-Side Comparison

| Aspect | Patch-based AMR (current) | Octree |
|---|---|---|
| Refinement granularity | whole rectangular patch | per-leaf (1 cell or block of cells) |
| Refinement decision | user, static | criterion-based, optionally adaptive |
| Tree depth | limited by user input (typically 1–3) | arbitrary; bounded only by `maxLevel` |
| Topology storage | flat array of grids + `motherP` | node pool with parent/child pointers |
| Cell → data lookup | $\mathcal{A}(i,j,k)$, $\mathcal{O}(1)$ | Morton-key binary search, $\mathcal{O}(\log N)$ |
| Cross-grid dispatch | sign test on $\mathcal{A}$, helper modules | implicit in tree descent |
| Ray traversal | 3D-DDA + sub-grid switch events | parametric octree, optionally with mini-DDA in blocks |
| MPI partition | flat round-robin | natural Morton/SFC partition |
| Output | grid-by-grid blocks | node table + leaf payload |
| Memory pattern | per-grid contiguous (1D `nCells`) | per-leaf contiguous; same shared-window infra reusable |
| Implementation cost | $\approx$0 (already in place) | 9–11 weeks core path (excluding adaptivity) |

## 5. Why Bother? When the Equivalence Breaks

For most existing MOCASSIN runs — one or two static sub-grids of moderate size — the patch design is perfectly adequate, and an octree would only add overhead. The case for replacement is concrete in three regimes:

1. *Multi-scale geometries*. When the resolution required for the ionisation front (or the Strömgren boundary, or a dust shadow boundary) varies by orders of magnitude across the domain, expressing this as rectangular patches forces large unrefined regions to inherit the finest resolution of any patch that overlaps them. Octree refinement is local.

2. *Unknown refinement location*. If the structure that needs resolving is not known a priori — for example, a moving shock front, or the contact discontinuity between a wind and the ambient medium — static patches cannot follow it. An octree with an adaptivity hook can.

3. *Deep nesting*. The current dispatch loop is bounded by `nGrids+2`, and the cost of a sub-grid switch is non-negligible. At nesting depths beyond three or four, the constant factors begin to matter; a unified tree-descent loop scales linearly with depth without this overhead.

None of these alone justifies a 2–3 month implementation effort, but in combination they argue for the move.

## 6. Implementation Sketch (cross-reference)

The detailed eight-phase plan, with file/line targets, regression strategy and effort estimates, is recorded separately in `mocassin-mocassin.2.02.73.2/OCTREE_PLAN.md`. Phases at a glance:

**O0** Five upstream design decisions (leaf size, tree representation, refinement criterion, dynamic vs. static, backwards compatibility).

**O1** New `octree_mod` with no call sites.

**O2** Read-time builder converting existing patch input to an octree; consistency assertions only.

**O3** Per-leaf data mirroring the legacy `nCells` arrays; invariant checks each iteration.

**O4** Octree ray traversal behind a feature flag; the present DDA path remains the truth source.

**O5** Optional dynamic refinement criterion.

**O6** Morton-SFC MPI redistribution.

**O7** Output format extension; backwards compatibility preserved.

**O8** Input keywords and final regression matrix.

The principle throughout is that each phase remains bit-identical to the previous on every existing regression case, and that the octree path is held to within statistical noise of the patch path on the same nine-case suite already exercised by Phase 1 and Phase 2.

# 7. Conclusion

The current MOCASSIN refinement design is a flat, statically-defined collection of rectangular sub-grids stitched together by a sign trick on the active-cell map. It is simple, fast, and adequate for most existing science cases. An octree replacement would unify the cross-grid dispatch into a single recursive descent, decouple refinement from user input, and expose a natural space-filling-curve partition for MPI — benefits that materialise only when the science calls for deep, multi-scale, or adaptively located refinement. The transition is well-scoped and incremental, but it is a real engineering investment; the decision to undertake it should be driven by a concrete science requirement rather than by the appeal of the data structure itself.